

Database Project Report

CS6083 Spring 2026
snickr: A Web-Based Collaboration System

Christine Wagner
caw561@nyu.edu

Hongyi Zeng
hz3866@nyu.edu

April 19, 2026

Introduction

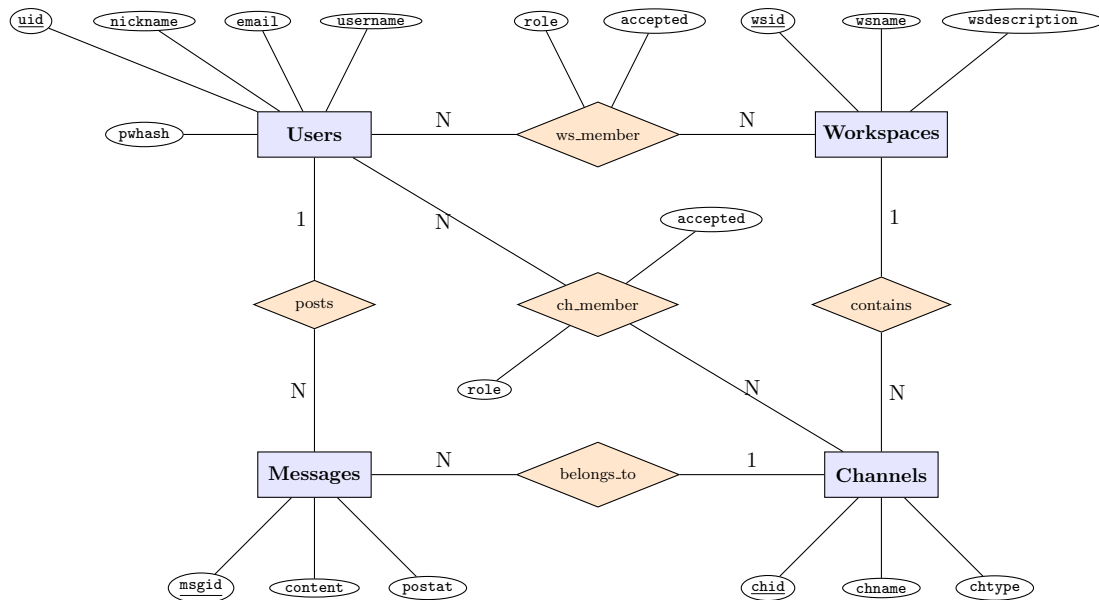
This report documents the design and implementation of *snickr*, a web-based collaboration system similar to Slack. The system allows users to create workspaces, organize communication through public, private, and direct channels, and exchange messages within those channels.

The database is built on PostgreSQL and consists of six relations: **users**, **workspaces**, **workspace_members**, **channels**, **channel_members**, and **messages**. Access control is enforced at the application level rather than through database permissions or views — a single DBMS account is used, and the application restricts content visibility based on workspace and channel membership.

This report covers the schema design with ER diagrams and relational translation (Section a), schema creation SQL (Section b), seven required queries with test results (Section c), and a discussion of the test data used to validate the queries (Section d).

(a) Database Schema Design

(a.1) ER Diagram



(a.2) Relational Schema

The ER diagram translates into the following six relations:

1. **users**(uid, email, username, nickname, pwhash, createdat, updatedat)
 - email is UNIQUE NOT NULL
2. **workspaces**(wsid, wsname, wsdescription, createdat, updatedat)
3. **workspace_members**(wsid, uid, role, accepted, createdat, updatedat)
 - $\text{wsid} \rightarrow \text{workspaces}(\text{wsid})$
 - $\text{uid} \rightarrow \text{users}(\text{uid})$
 - $\text{role} \in \{\text{creator, admin, member}\}$
4. **channels**(chid, wsid, chname, chtype, createdat, updatedat)
 - $\text{wsid} \rightarrow \text{workspaces}(\text{wsid})$
 - $\text{chtype} \in \{\text{public, private, direct}\}$
5. **channel_members**(chid, uid, role, accepted, createdat, updatedat)
 - $\text{chid} \rightarrow \text{channels}(\text{chid})$
 - $\text{uid} \rightarrow \text{users}(\text{uid})$
 - $\text{role} \in \{\text{creator, member}\}$
6. **messages**(msgid, chid, uid, content, postat)
 - $\text{chid} \rightarrow \text{channels}(\text{chid})$
 - $\text{uid} \rightarrow \text{users}(\text{uid})$

(a.3) Design Assumptions

- **A surrogate uid is used as the primary key for users**, rather than email. This allows users to change their email address without cascading updates across all foreign keys. The email column is constrained as UNIQUE NOT NULL.
- **Creator is stored in membership tables, not in workspace/channel tables.** The creator is the first member inserted with role = ‘creator’. As specified in the requirements, the creator is treated as the initial administrator; query (3) therefore includes both **creator** and **admin** roles when listing administrators. Storing the creator as a role avoids redundancy and simplifies queries.
- **Invitations are tracked via the accepted field**, not as a separate entity. When a user is invited to a workspace or channel, a membership record is inserted with **accepted** = FALSE. Upon acceptance, it is updated to TRUE. As a trade-off, the system does not track *who* sent the invitation — only whether the user has accepted.
- **Public channels are visible to all workspace members but still require explicit membership.** Users are not automatically added upon channel creation; instead, they must choose to join. When a user joins a public channel, a record is inserted into **channel_members** with **accepted** = TRUE, granting access to the channel’s messages.

- **Passwords are stored as hashes.** The `pwhash` field stores a hashed password rather than plaintext for security.
- **Timestamps are recorded for all actions.** All tables include `createdat` and `updatedat` fields (messages use `postat` instead). While `updatedat` is not exercised by the required queries, it is included to support future features such as profile edits, role changes, and invitation acceptance tracking.
- **Member removal** is supported by deleting the corresponding row from `workspace_members` or `channel_members`. No soft-delete flag is needed since the membership record itself represents the relationship.
- **Direct channels are limited to two members**, enforced at the application level rather than via a database constraint, since a `CHECK` on member count would require a trigger.
- **Access control is enforced at the application level**, not through database permissions or views, as specified in the project requirements.

(a.4) Application-Level Enforcement

The following constraints are enforced by the web application rather than the database, either because SQL `CHECK` constraints cannot express them without triggers, or because they involve session state:

- **Direct channel membership limit.** The application ensures that exactly two users are associated with a direct channel at creation time. Enforcing this constraint at the database level would require counting rows across tables, which is not possible with a simple `CHECK` constraint.
- **Workspace invitation authority.** Only users with `role` IN ('creator', 'admin') in `workspace_members` are permitted to insert new membership records for other users.
- **Private channel invitation authority.** Only the channel creator (`role` = 'creator' in `channel_members`) may invite additional users to a private channel.
- **Session identification.** In Part 2, each HTTP request will include a session cookie identifying the logged-in user `uid`. The application resolves this identity before executing any query, ensuring that users can only access or modify data on their own behalf.
- **Public channel join eligibility.** Before inserting a `channel_members` row for a public channel, the application verifies that the user is an accepted member of the corresponding workspace. This enforces cross-table consistency that is not captured by foreign key constraints alone.

(a.5) Space Efficiency

- **No redundant creator columns.** Instead of storing a separate `creator_uid` in the `workspaces` and `channels` tables, we encode the creator as a membership record with `role` = 'creator'. This avoids duplicating user references.
- **Composite primary keys** on `workspace_members` and `channel_members` eliminate the need for surrogate ID columns in junction tables.

- **VARCHAR with appropriate limits** is used instead of fixed-length **CHAR** fields, so shorter values do not waste space.
- **SERIAL auto-increment** is used for `uid`, `wsid`, `chid`, and `msgid` (4-byte integers), which is more compact than UUIDs. Using an integer `uid` as the user primary key also makes foreign key columns in membership and message tables 4 bytes instead of up to 255 bytes for an email string.
- **No separate invitations table.** The `accepted` boolean field on membership tables tracks invitation state in-place, avoiding an extra table and the joins it would require.

(b) Schema Creation

The following SQL creates the schema in PostgreSQL with all key, foreign key, and check constraints:

```
CREATE TABLE users (
    uid          SERIAL          PRIMARY KEY,
    email        VARCHAR(255)    NOT NULL UNIQUE,
    username     VARCHAR(50)     NOT NULL,
    nickname     VARCHAR(50),
    pwhash       VARCHAR(255)    NOT NULL,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspaces (
    wsid         SERIAL          PRIMARY KEY,
    wsname       VARCHAR(100)    NOT NULL,
    wsdescription TEXT,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspace_members (
    wsid         INT             NOT NULL REFERENCES workspaces(wsid),
    uid          INT             NOT NULL REFERENCES users(uid),
    role         VARCHAR(10)     NOT NULL CHECK (role IN ('creator', 'admin', 'member')),
    accepted     BOOLEAN         NOT NULL DEFAULT FALSE,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (wsid, uid)
);

CREATE TABLE channels (
    chid         SERIAL          PRIMARY KEY,
    wsid         INT             NOT NULL REFERENCES workspaces(wsid),
    chname       VARCHAR(100),
    chtype       VARCHAR(10)     NOT NULL CHECK (chtype IN ('public', 'private', 'direct')),
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE channel_members (
    chid         INT             NOT NULL REFERENCES channels(chid),
    uid          INT             NOT NULL REFERENCES users(uid),
    role         VARCHAR(10)     NOT NULL CHECK (role IN ('creator', 'member')),
    accepted     BOOLEAN         NOT NULL DEFAULT FALSE,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (chid, uid)
);

CREATE TABLE messages (
    msgid        SERIAL          PRIMARY KEY,
    chid         INT             NOT NULL REFERENCES channels(chid),
    uid          INT             NOT NULL REFERENCES users(uid),
    content      TEXT            NOT NULL,
    postat       TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

(c) Queries

(1) Create a new user account

```
CREATE OR REPLACE PROCEDURE create_user(  
    p_email    VARCHAR,  
    p_username VARCHAR,  
    p_nickname VARCHAR,  
    p_pwhash   VARCHAR  
)  
LANGUAGE SQL  
AS $$  
    INSERT INTO users (email, username, nickname, pwhash)  
    VALUES (p_email, p_username, p_nickname, p_pwhash);  
$$;  
  
-- Example call:  
CALL create_user('gz1234@nyu.edu', 'gracezhu', 'Grace', 'a1b2c3d4e5f6');
```

Test: Calling with a new email succeeds and inserts a row into `users`. Calling with a duplicate email raises a unique constraint violation.

(2) Create a new public channel (with authorization check)

```
CREATE OR REPLACE PROCEDURE create_public_channel(  
    p_wsuid INT,  
    p_uid   INT,  
    p_chname VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_chid INT;  
BEGIN  
    -- Check if the user is an accepted member of the workspace  
    IF NOT EXISTS (  
        SELECT 1 FROM workspace_members  
        WHERE wsid = p_wsuid AND uid = p_uid AND accepted = TRUE  
    ) THEN  
        RAISE EXCEPTION 'User % is not an accepted member of workspace %',  
            p_uid, p_wsuid;  
    END IF;  
  
    -- Create the public channel  
    INSERT INTO channels (wsid, chname, chtype)  
    VALUES (p_wsuid, p_chname, 'public')  
    RETURNING chid INTO v_chid;  
  
    -- Add the creator as a channel member  
    INSERT INTO channel_members (chid, uid, role, accepted)  
    VALUES (v_chid, p_uid, 'creator', TRUE);  
END;  
$$;  
  
-- Example call:  
CALL create_public_channel(1, 1, 'announcements');
```

Test: Calling with an accepted workspace member succeeds, creating a new channel and adding the user as channel creator. Calling with a non-member (e.g., Eve, who has a pending invite) raises the exception `User % is not an accepted member of workspace %`.

(3) For each workspace, list all current administrators

```
SELECT w.wsid, w.wsname, u.email, u.username, wm.role
FROM workspaces w
JOIN workspace_members wm ON w.wsid = wm.wsid
JOIN users u ON wm.uid = u.uid
WHERE wm.role IN ('creator', 'admin') AND wm.accepted = TRUE
ORDER BY w.wsid, wm.role;
```

Result:

wsid	wsname	email	username	role
1	CS Department	bs5678@nyu.edu	bobsmith	admin
1	CS Department	az1234@nyu.edu	alicezhang	creator
2	Tandon Robotics Club	cw9012@nyu.edu	carolwang	admin
2	Tandon Robotics Club	bs5678@nyu.edu	bobsmith	creator

(4) Pending invitations per public channel (older than 5 days)

```
SELECT c.chid, c.chname, COUNT(cm.uid) AS pending_count
FROM channels c
LEFT JOIN channel_members cm ON c.chid = cm.chid
AND cm.accepted = FALSE
AND cm.createdat < CURRENT_TIMESTAMP - INTERVAL '5 days'
WHERE c.wsid = 1 AND c.chtype = 'public'
GROUP BY c.chid, c.chname
ORDER BY c.chid;
```

Result:

chid	chname	pending_count
1	general	1

(5) List all messages in a channel in chronological order

```
CREATE OR REPLACE FUNCTION list_channel_messages(p_chid INT)
RETURNS TABLE (
    msgid INT, email VARCHAR, username VARCHAR,
    content TEXT, postat TIMESTAMP
)
LANGUAGE SQL
AS $$
    SELECT m.msgid, u.email, u.username, m.content, m.postat
    FROM messages m
    JOIN users u ON m.uid = u.uid
    WHERE m.chid = p_chid
    ORDER BY m.postat ASC;
$$;

-- Example call:
SELECT * FROM list_channel_messages(1);
```

Result:

msgid	email	username	content	postat
1	az1234@nyu.edu	alicezhang	Welcome to the CS Department workspace!	2026-04-01 09:00
2	bs5678@nyu.edu	bobsmith	Thanks Alice! Excited to be here.	2026-04-01 09:05
3	cw9012@nyu.edu	carolwang	Is the new curriculum perpendicular to the old one?	2026-04-01 09:10
4	az1234@nyu.edu	alicezhang	Not exactly perpendicular, but quite different.	2026-04-01 09:15

(6) List all messages posted by a particular user

```
SELECT m.msgid, c.chname, m.content, m.postat
FROM messages m
JOIN channels c ON m.chid = c.chid
WHERE m.uid = 1 -- Alice
ORDER BY m.postat DESC;
```

Result:

msgid	chname	content	postat
7	direct	Bob, can you handle the seminar logistics?	2026-04-03 14:00
5	hiring-committee	We need to review the new candidates.	2026-04-02 10:00
4	general	Not exactly perpendicular, but quite different.	2026-04-01 09:15
1	general	Welcome to the CS Department workspace!	2026-04-01 09:00

(7) Search accessible messages by keyword

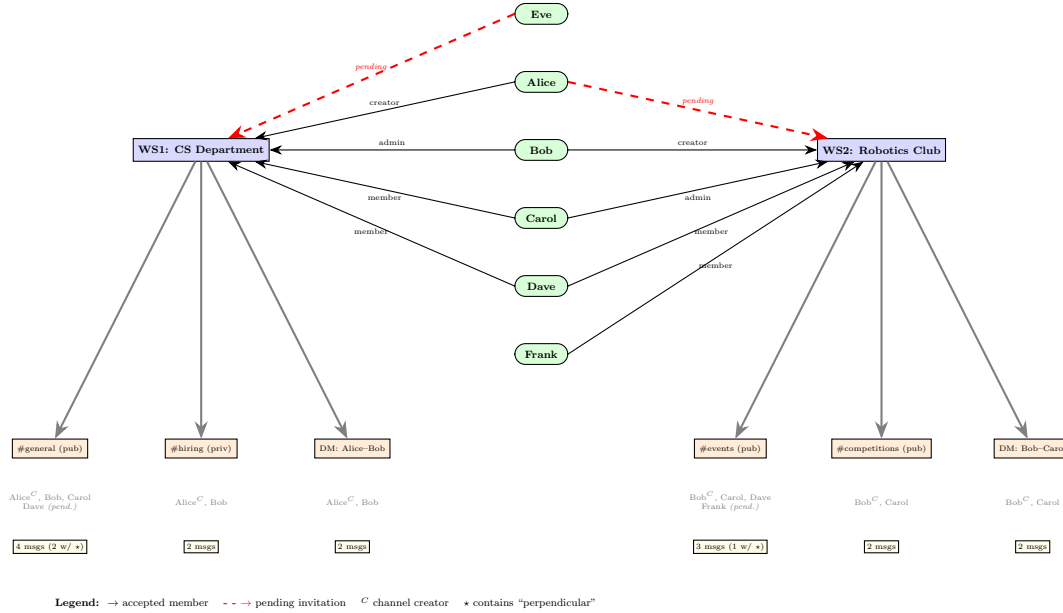
```
CREATE OR REPLACE FUNCTION search_accessible_messages(
    p_uid INT, p_keyword VARCHAR
)
RETURNS TABLE (
    msgid INT, chname VARCHAR, author VARCHAR,
    content TEXT, postat TIMESTAMP
) LANGUAGE SQL AS $$
    SELECT m.msgid, c.chname, u.email AS author,
           m.content, m.postat
    FROM messages m
    JOIN channels c ON m.chid = c.chid
    JOIN users u ON m.uid = u.uid
    JOIN channel_members cm ON c.chid = cm.chid AND cm.uid = p_uid
    JOIN workspace_members wm
        ON c.wsid = wm.wsid AND wm.uid = p_uid
    WHERE cm.accepted = TRUE
        AND wm.accepted = TRUE
        AND m.content ILIKE '%' || p_keyword || '%'
    ORDER BY m.postat DESC;
$$;
-- Example call:
SELECT * FROM search_accessible_messages(1, 'perpendicular');
```

Result:

msgid	chname	author	content	postat
4	general	az1234@nyu.edu	Not exactly perpendicular, but quite different.	2026-04-01 09:15
3	general	cw9012@nyu.edu	Is the new curriculum perpendicular to the old one?	2026-04-01 09:10

(d) Test Data

The diagram below illustrates the structure of our test data. Solid arrows represent accepted memberships; dashed arrows represent pending invitations. Messages containing the keyword “perpendicular” are marked with ★.



Sample Data Snapshot

Users:

uid	email	username	nickname
1	az1234@nyu.edu	alicezhang	Alice
2	bs5678@nyu.edu	bobsmith	Bob
3	cw9012@nyu.edu	carolwang	Carol
4	dj3456@nyu.edu	davejohnson	Dave
5	ec7890@nyu.edu	evechen	Eve
6	fl2345@nyu.edu	frankli	Frank

Workspace Members (excerpt):

wsid	uid	role	accepted	notes
1	1	creator	TRUE	Alice created CS Dept
1	2	admin	TRUE	Bob is admin
1	5	member	FALSE	Eve invited, not joined
2	2	creator	TRUE	Bob created Robotics
2	1	member	FALSE	Alice invited, not joined

Data Structure Discussion

- **6 users** span two workspaces with overlapping membership: Alice, Bob, Carol, and Dave appear in both, while Eve and Frank each belong to only one — ensuring cross-workspace queries return correct results.
- **2 workspaces** each contain 3 channels covering all three types (public, private, direct), so queries that filter by channel type are exercised.

- **Role diversity:** Alice is a workspace creator and channel creator; Bob is both a workspace creator (Robotics Club) and an admin (CS Department). This tests query (3) which lists administrators.
- **Pending invitations:** Eve has not accepted her workspace invite; Dave and Frank have not accepted their channel invites (created >5 days ago). This tests query (4).
- **15 messages** are distributed across all 6 channels. Three messages contain the keyword “perpendicular” in two different channels across both workspaces, testing query (7)’s keyword search with access control.

Access Control Testing (Query 7)

We tested `search_accessible_messages` with the keyword “perpendicular” under four edge cases to verify that both workspace and channel membership conditions are enforced:

1. **Alice (uid=1):** Accepted member of workspace 1 and channel 1 (`#general`), which contains two messages with the keyword. *Result: 2 rows returned.*
2. **Dave (uid=4):** Accepted member of both workspaces. In channel 1 (`#general`), Dave has `accepted = FALSE` (pending); in channel 4 (`#events`), Dave is accepted. Both channels contain “perpendicular” messages. *Result: 1 row returned* — only the `#events` message, confirming channel-level access control blocks the pending channel.
3. **Bob (uid=2):** Accepted member of both workspaces and multiple channels. Channel 1 and channel 4 each contain keyword matches. *Result: 3 rows returned* (2 from `#general`, 1 from `#events`).
4. **Eve (uid=5):** Invited to workspace 1 but has `accepted = FALSE` (pending workspace invitation), so no channels are accessible. *Result: 0 rows returned.*